

GROUP #14

RICCARDO
COLLETTI

ALEXANDER
REMMERIE

XUTAO
MA

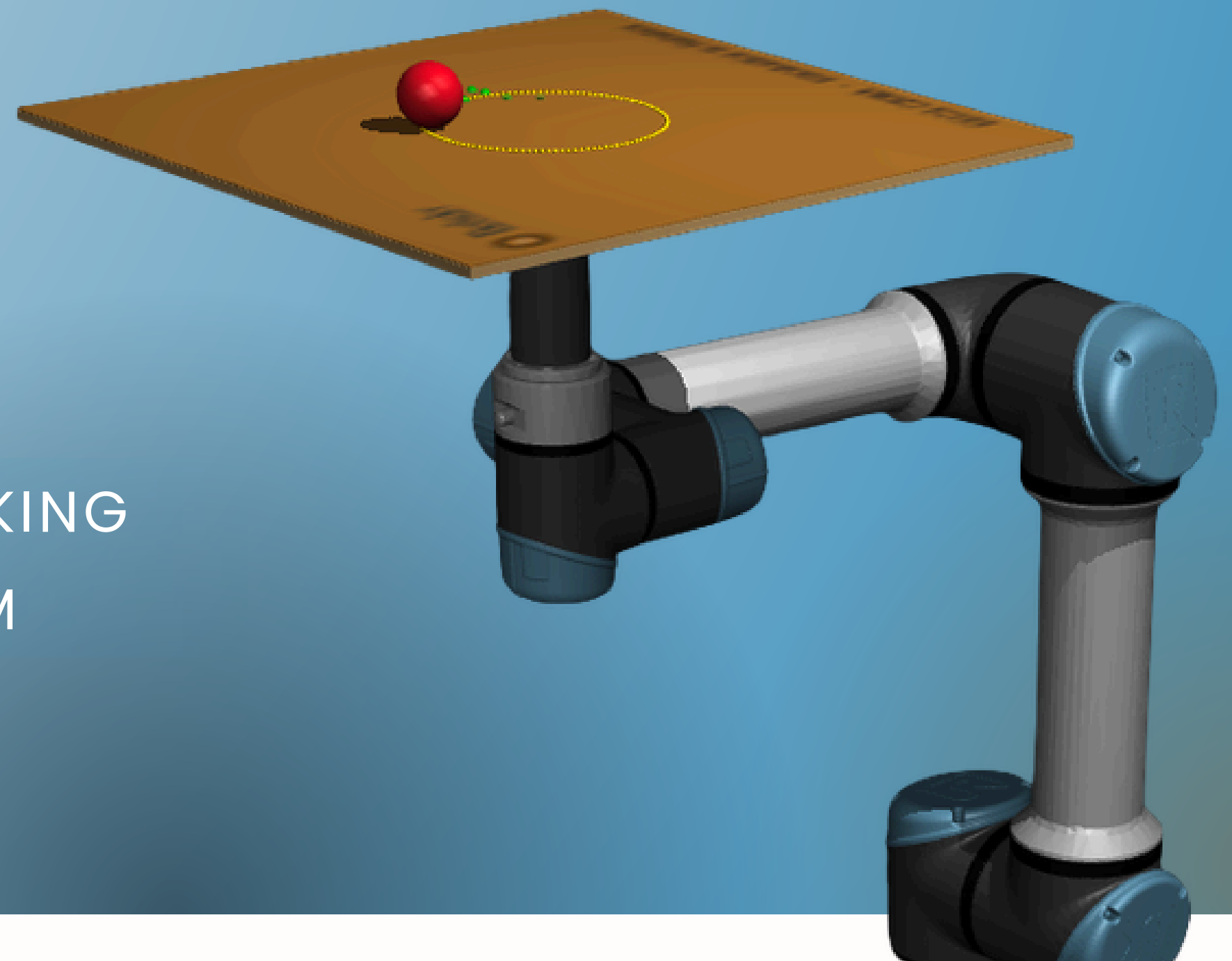
ALEXANDER
GASCA ROSAS

LARS
TALIAN

EECS206A

HOLD UR PLATE

VISION-BASED TRAJECTORY TRACKING
ON A UR7E BALL-ON-PLATE SYSTEM



PROJECT OVERVIEW

01

PERCEPTION

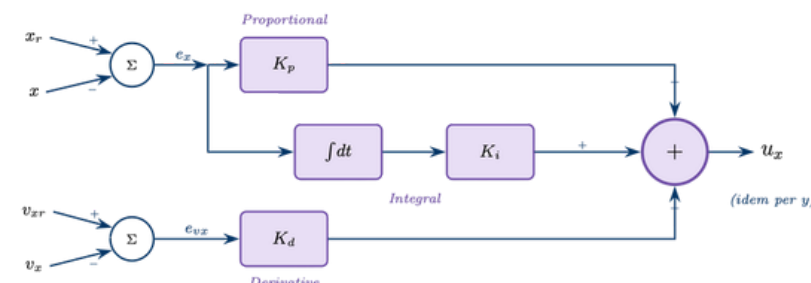
- RealSense D435i camera
- ArUco markers + homography
- HSV threshold (ball detection)
- Kalman filter $\rightarrow (x, y, \dot{x}, \dot{y})$



02

CONTROL

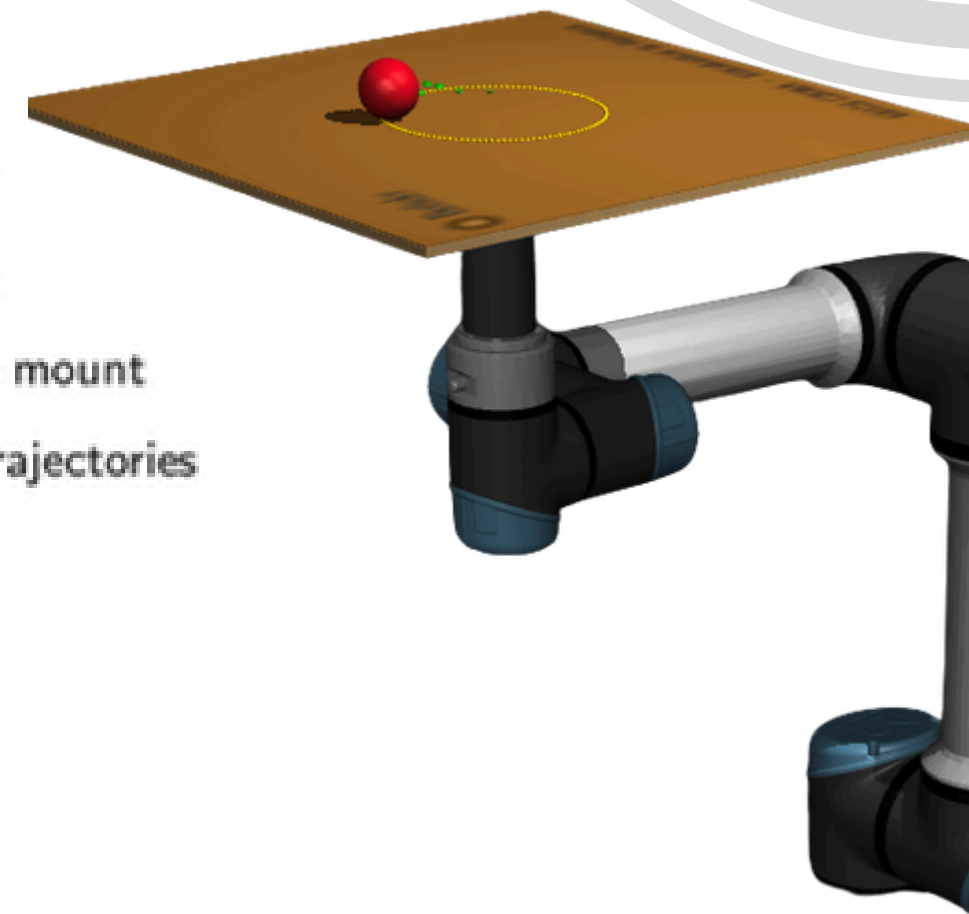
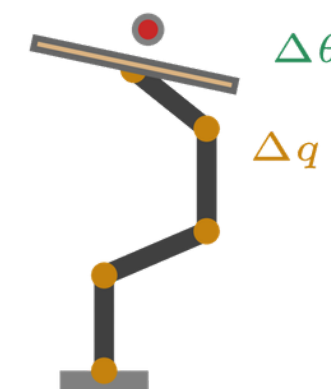
- PID per axis
- MPC (linearised dynamics)
- PPO policy (trained in MuJoCo)

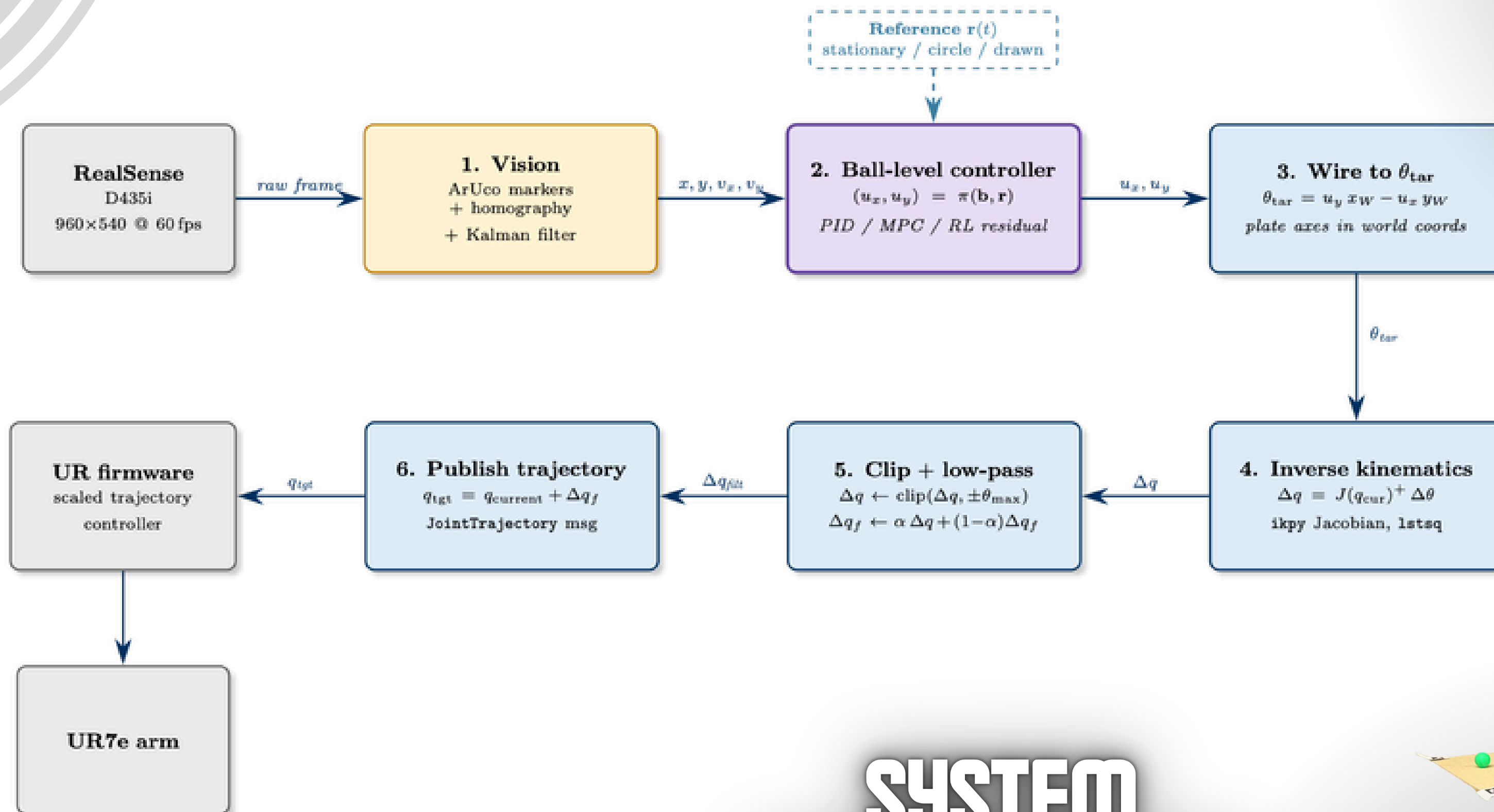


03

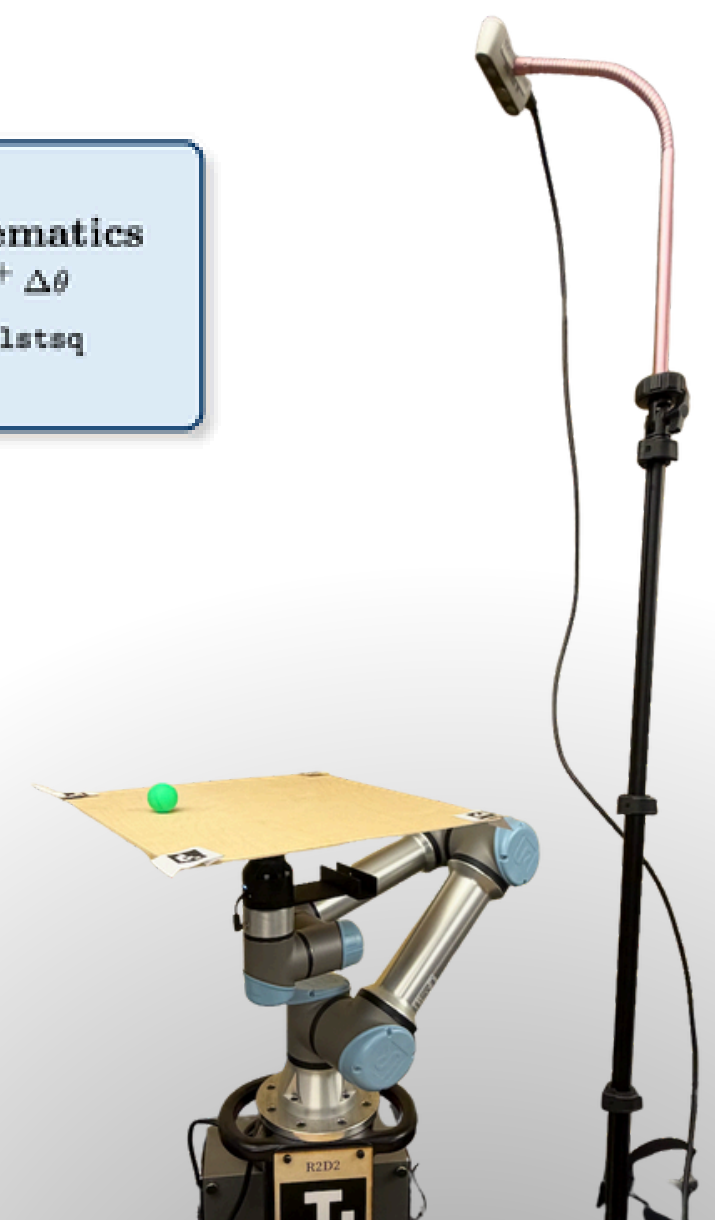
ACTUATION

- UR7e end-effector tilt
- Custom plate + plate mount
- `ros2_control` joint trajectories



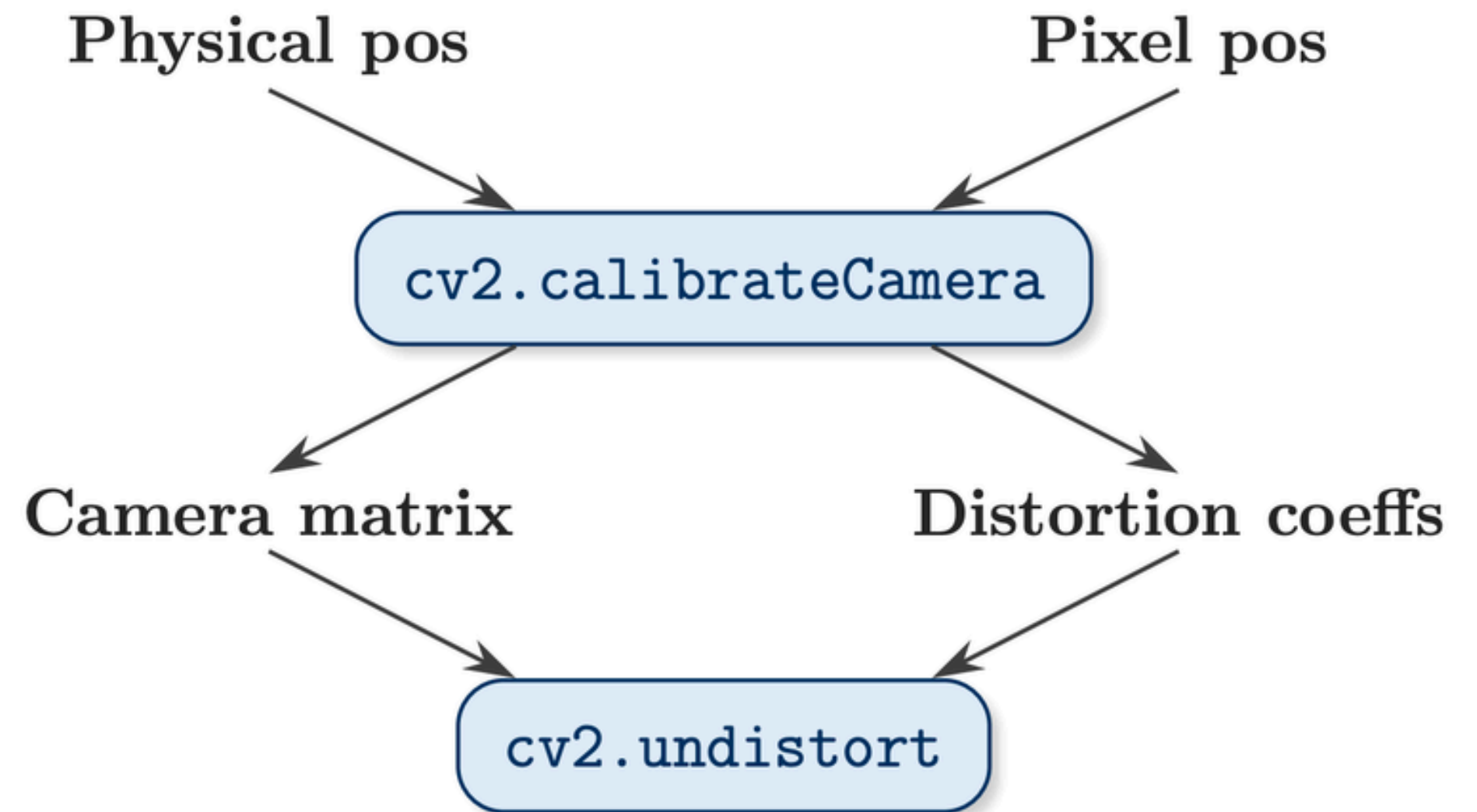
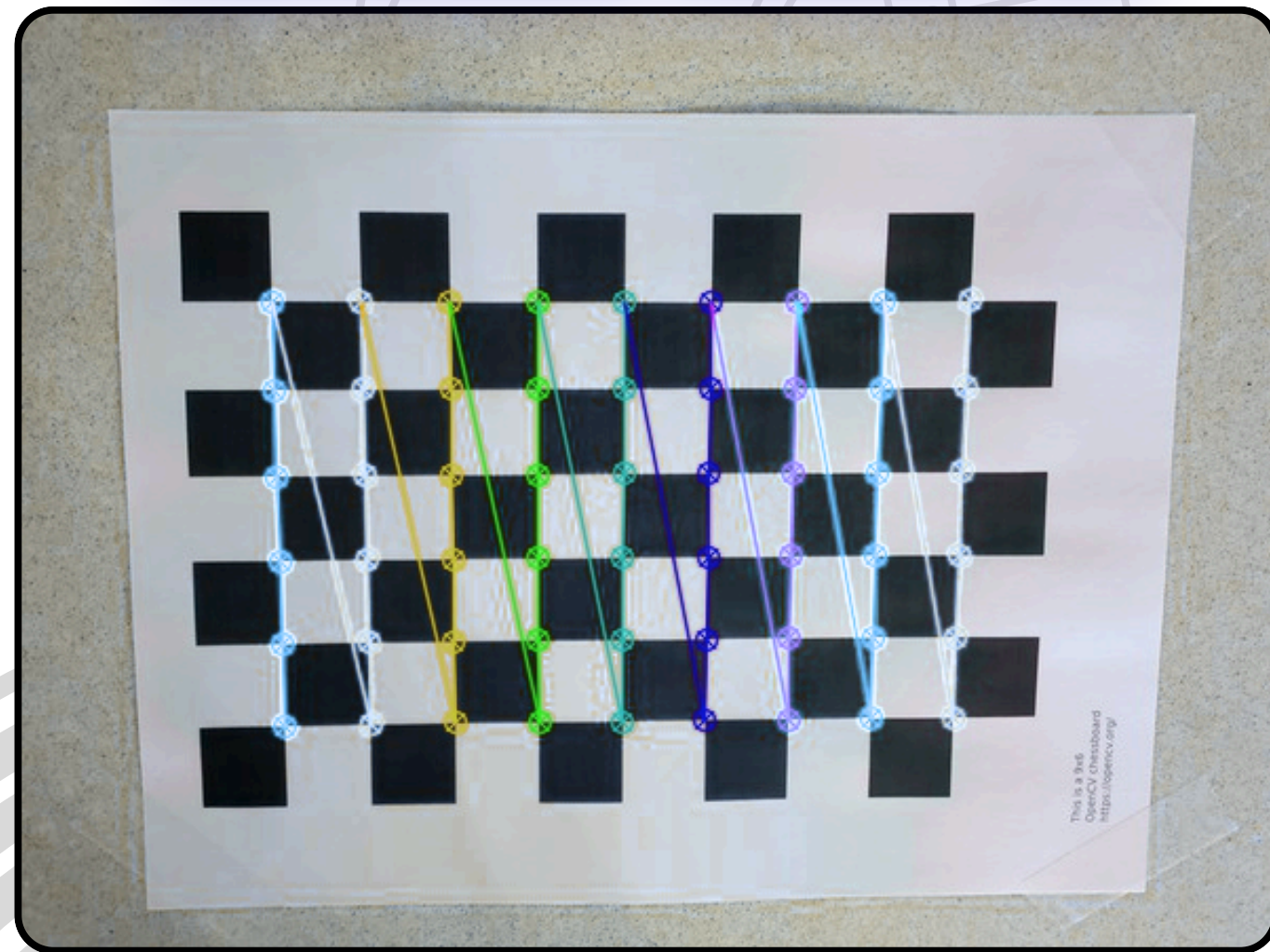


SYSTEM ARCHITECTURE



CAMERA CALIBRATION

CHECKBOARD CALIBRATION



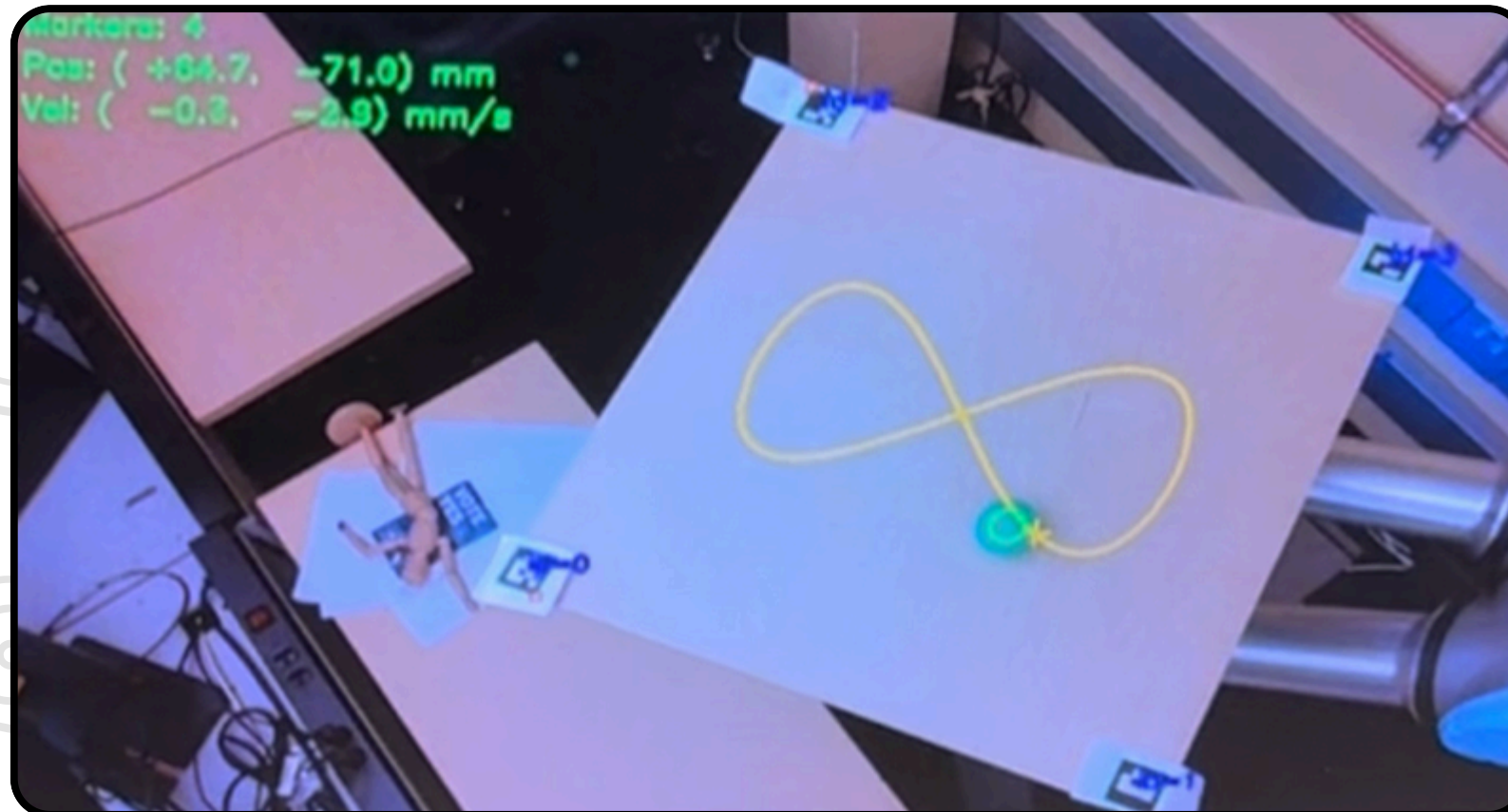
PERCEPTION

BALL TRACKER

Position detection:

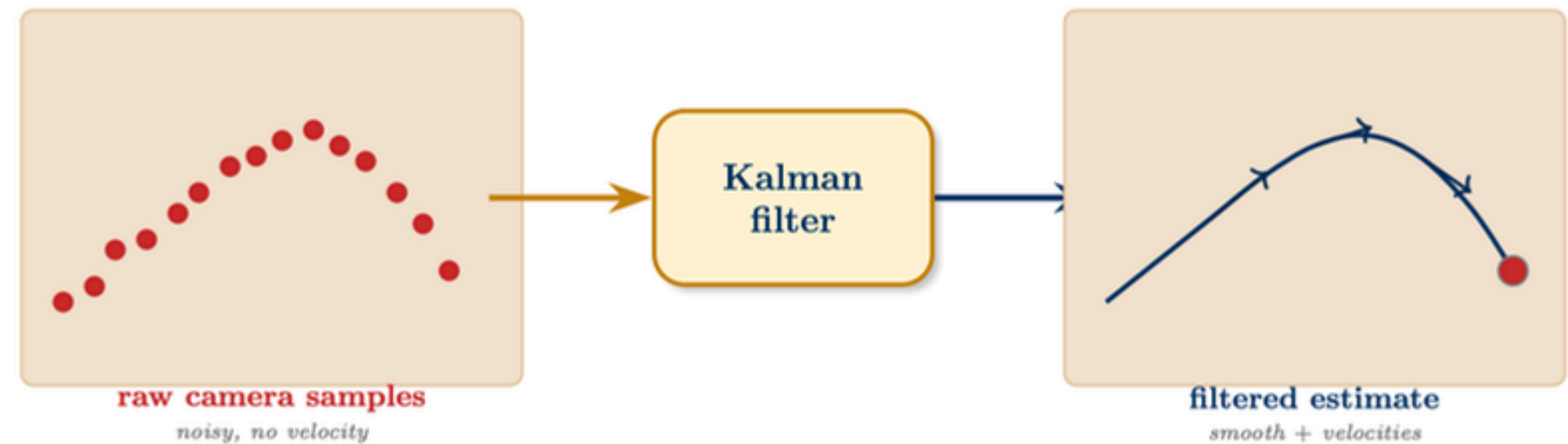
Threshold HSV:

1. Robust to lighting changes.
2. Isolates ball-colored (green) pixels into a binary mask.



PERCEPTION

BALL TRACKER



Velocity estimation:

Kalman filtering

System dynamics $\dot{X}_t = AX_t + Bu_t + \omega_t, Y_t = CX_t$ $X_t = [x_t \ y_t \ \dot{x}_t \ \dot{y}_t]^T$

Luenberger observer $\dot{\hat{X}}_t = A\hat{X}_t + Bu_t + L(Y_t - C\hat{X}_t)$

Estimation error $e = X_t - \hat{X}_t$ $\dot{e} = (A - LC)e + \omega$

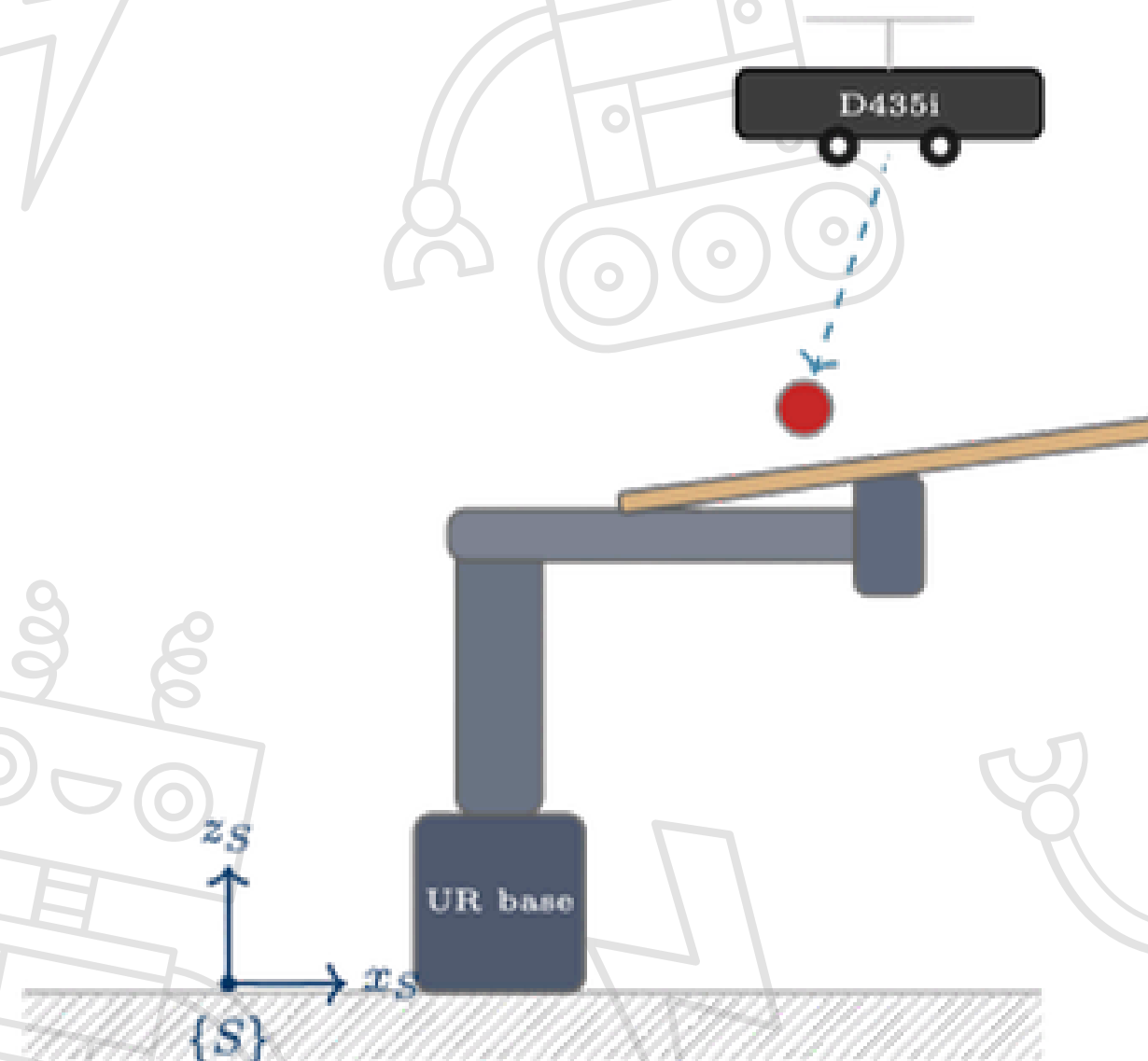
Minimize $E_{\omega} \|e\|_2^2$

PERCEPTION

SETUP

The rig (side view)

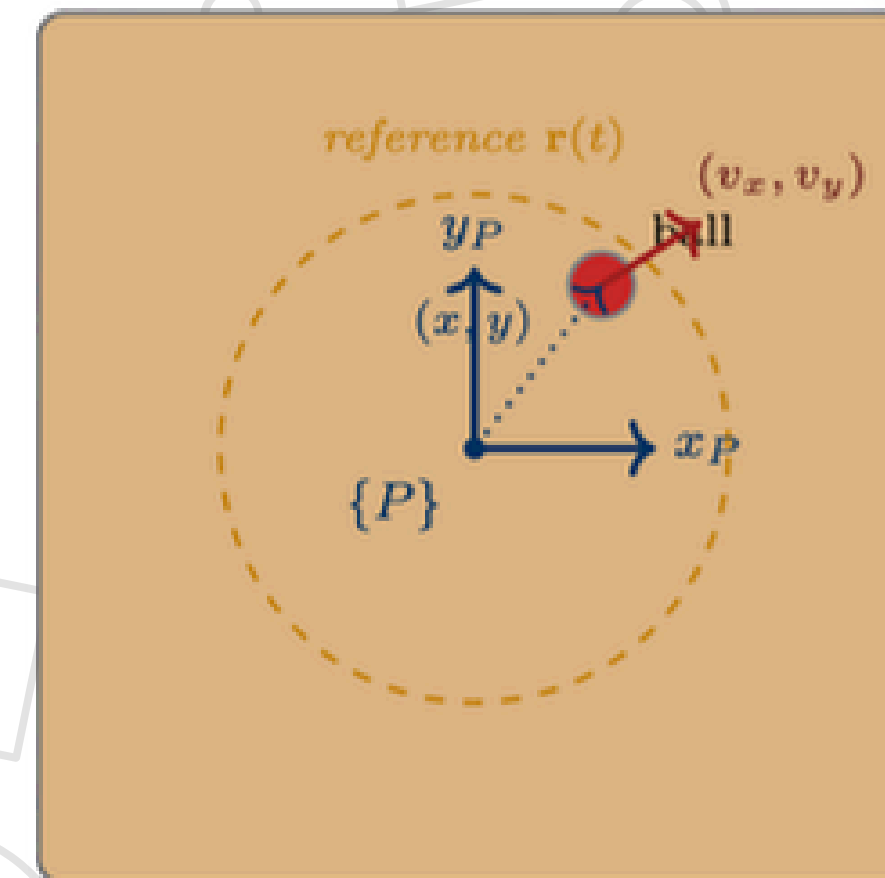
UR7e + plate + ball + RealSense D435i



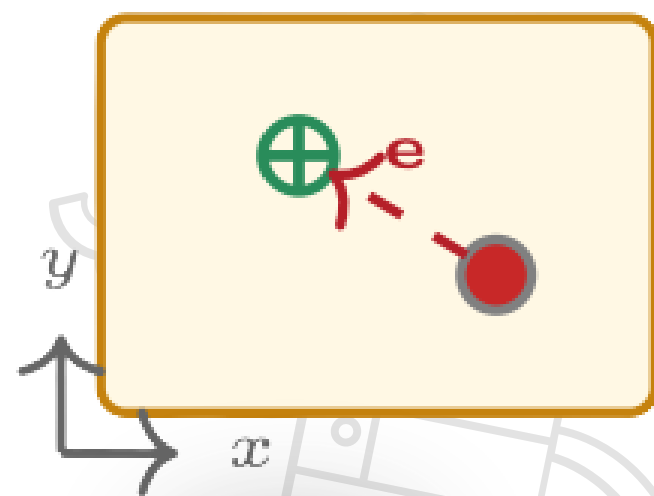
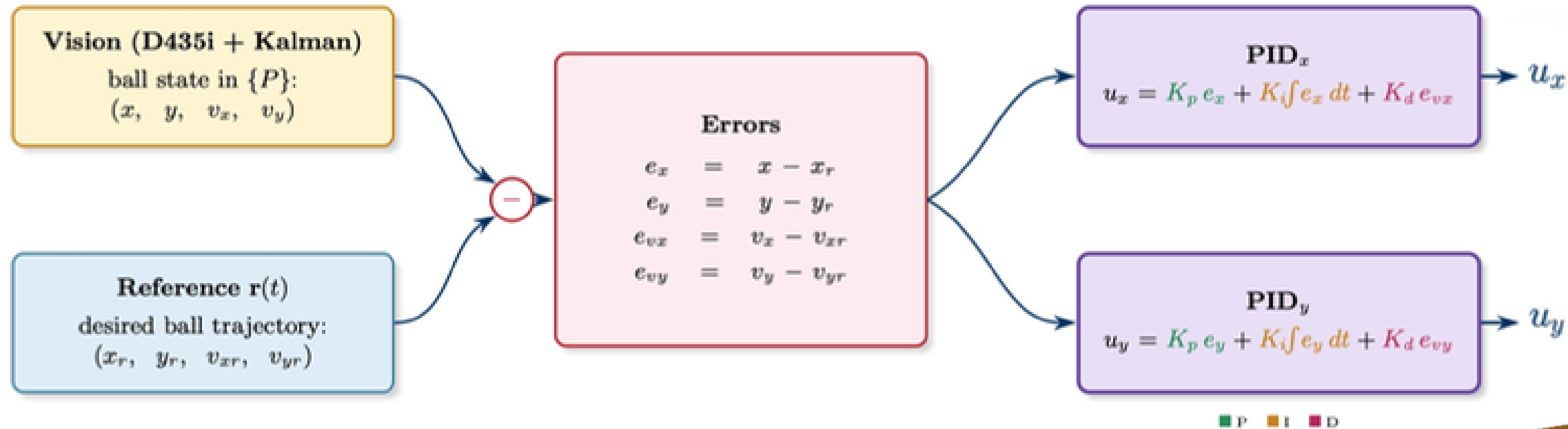
Top view: what we measure

ball state (x, y, v_x, v_y) in plate frame $\{P\}$

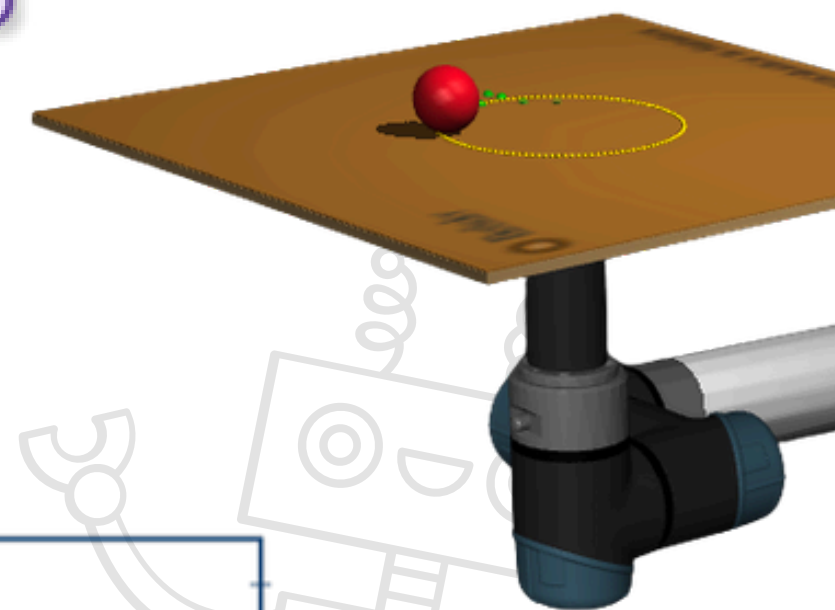
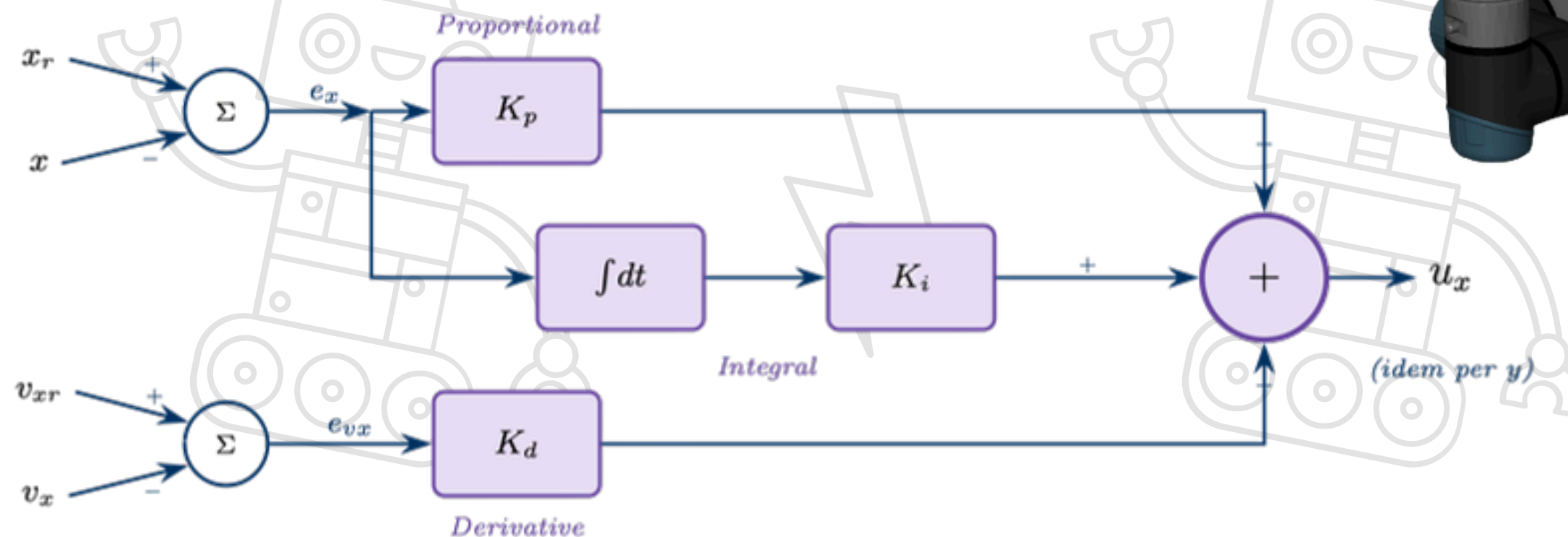
(camera looks straight down)



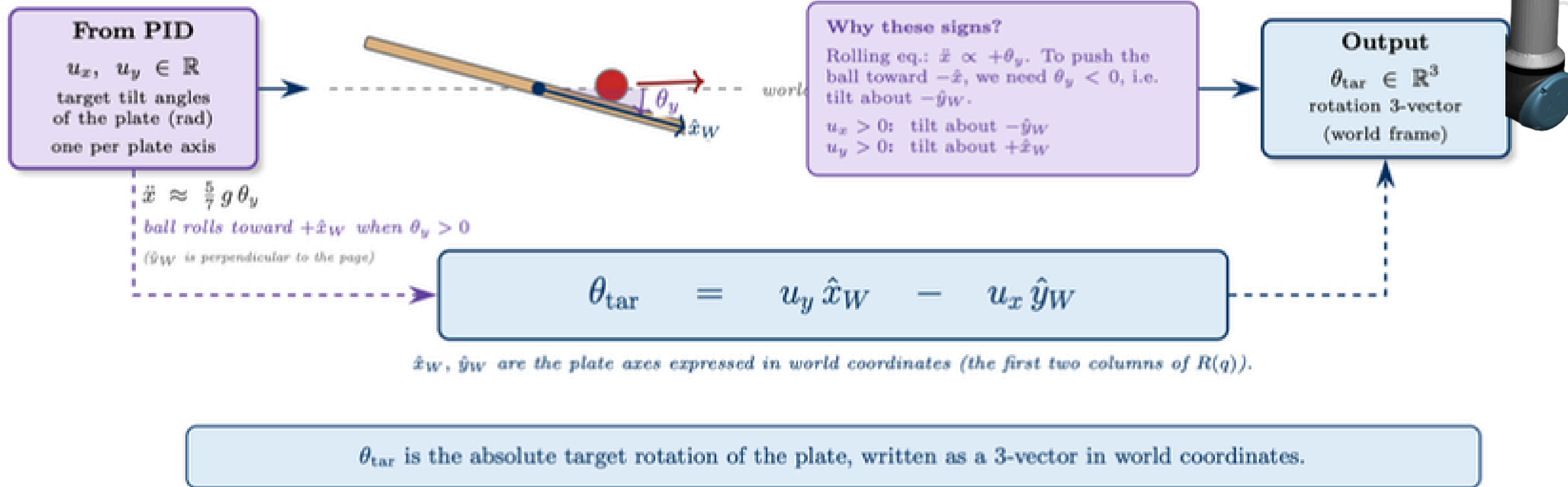
Goal: the ball must follow a reference trajectory $(x_r(t), y_r(t))$ defined on the plate, by tilting the plate through small wrist rotations.



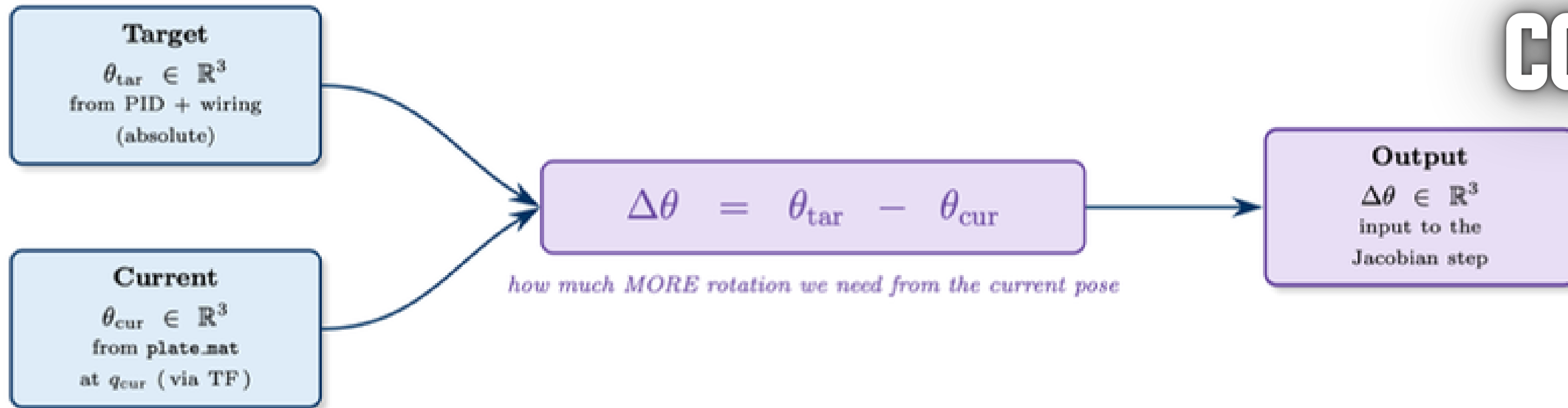
PID CONTROLLER



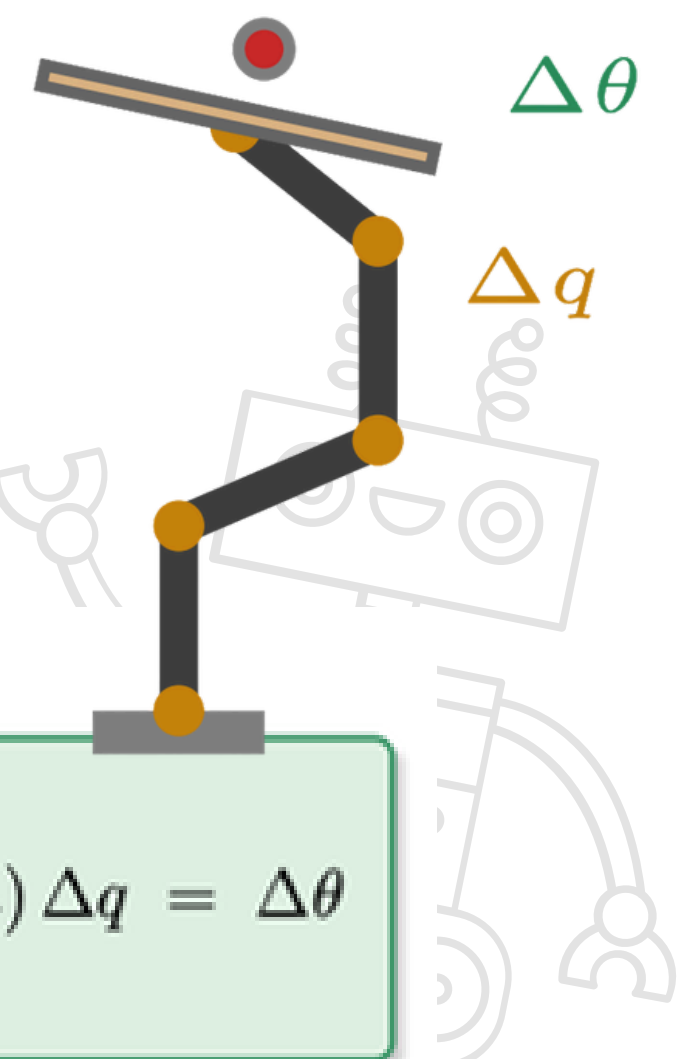
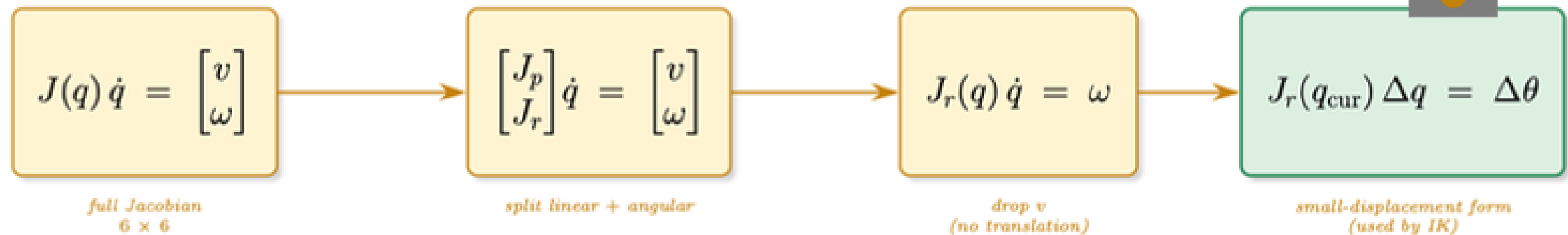
PID CONTROLLER



PID CONTROLLER

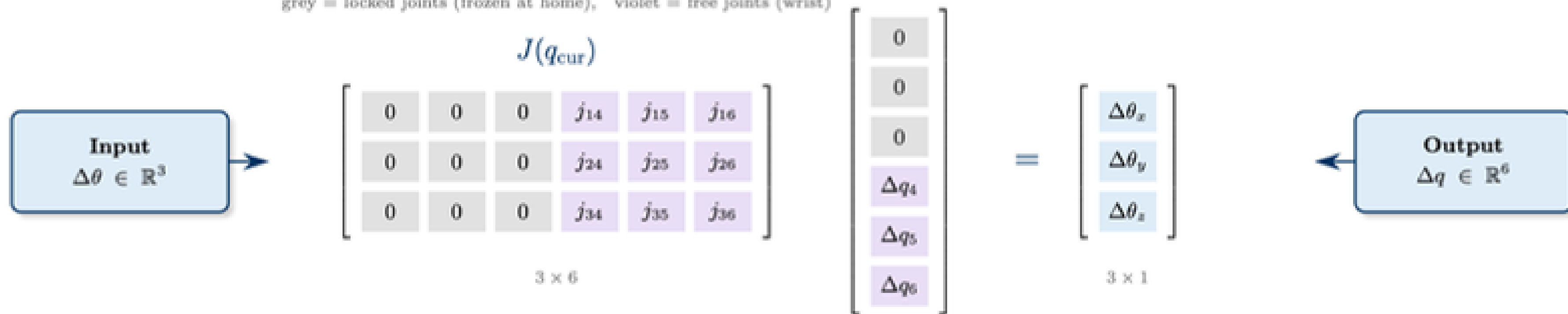


INVERSE KINEMATICS

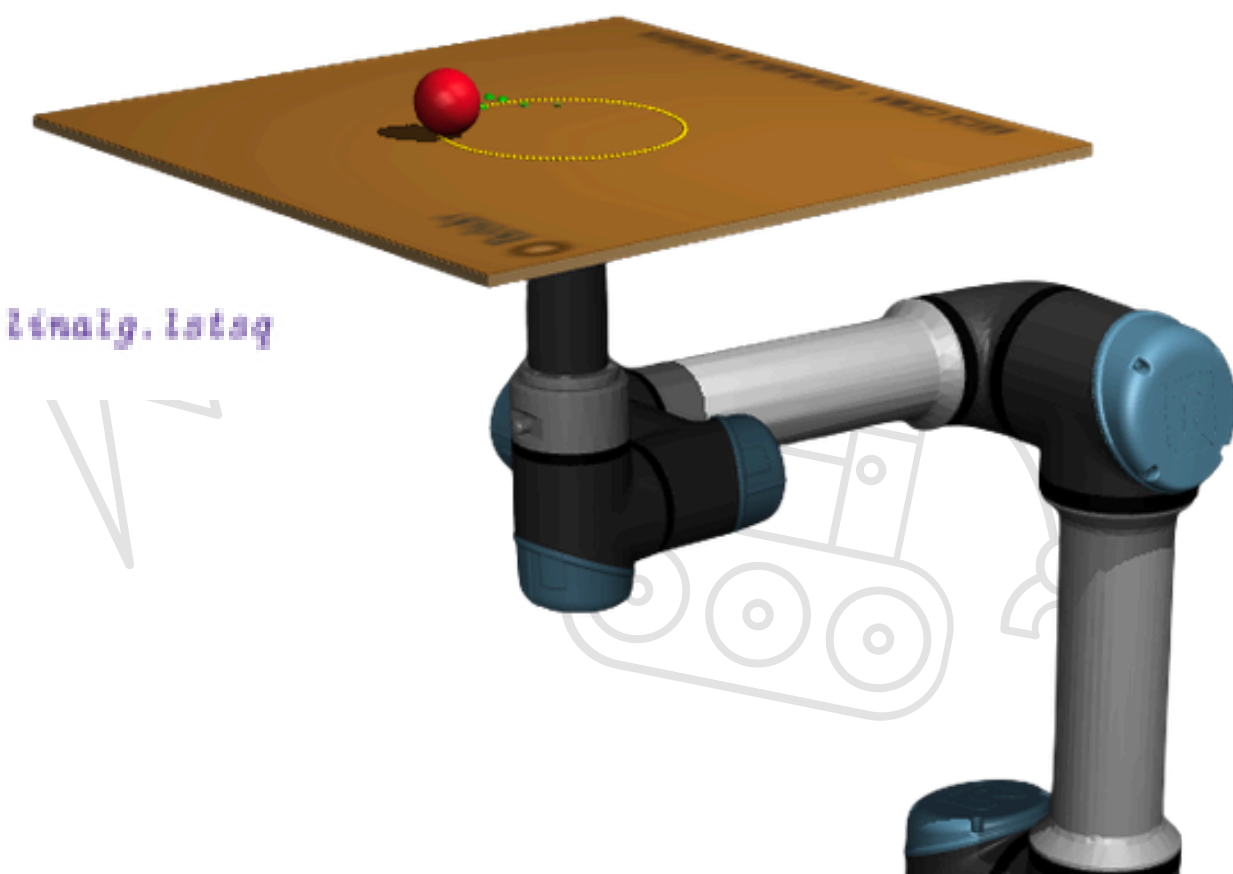


ACTUATION

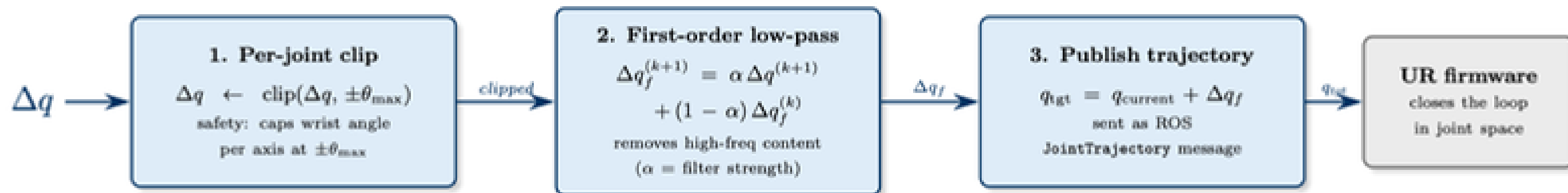
grey = locked joints (frozen at home), violet = free joints (wrist)



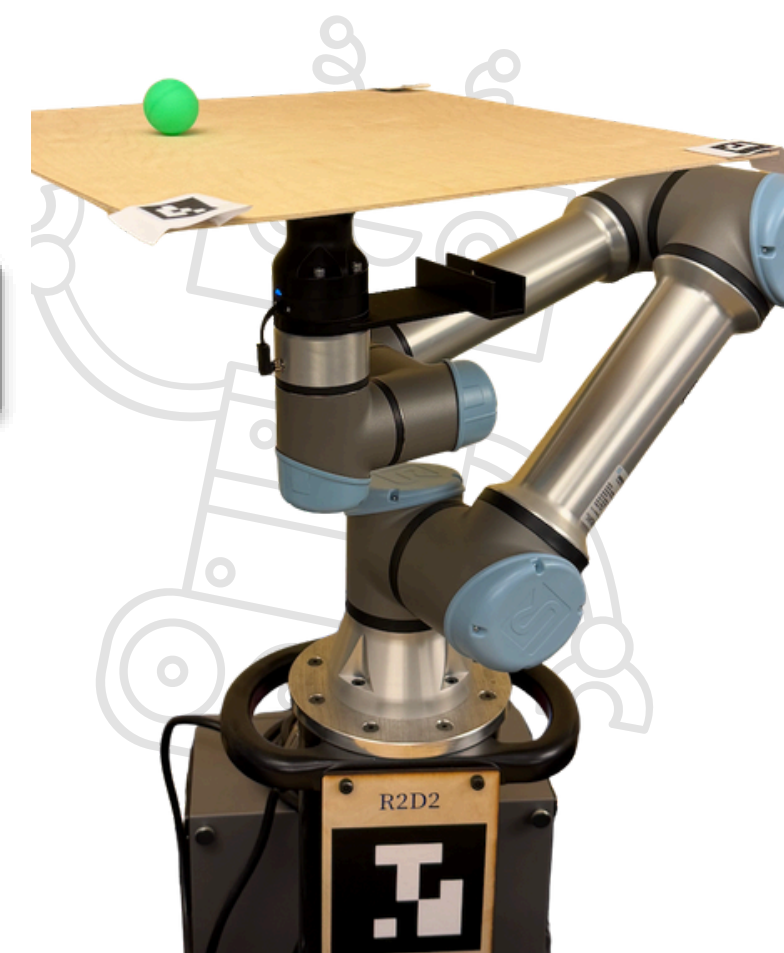
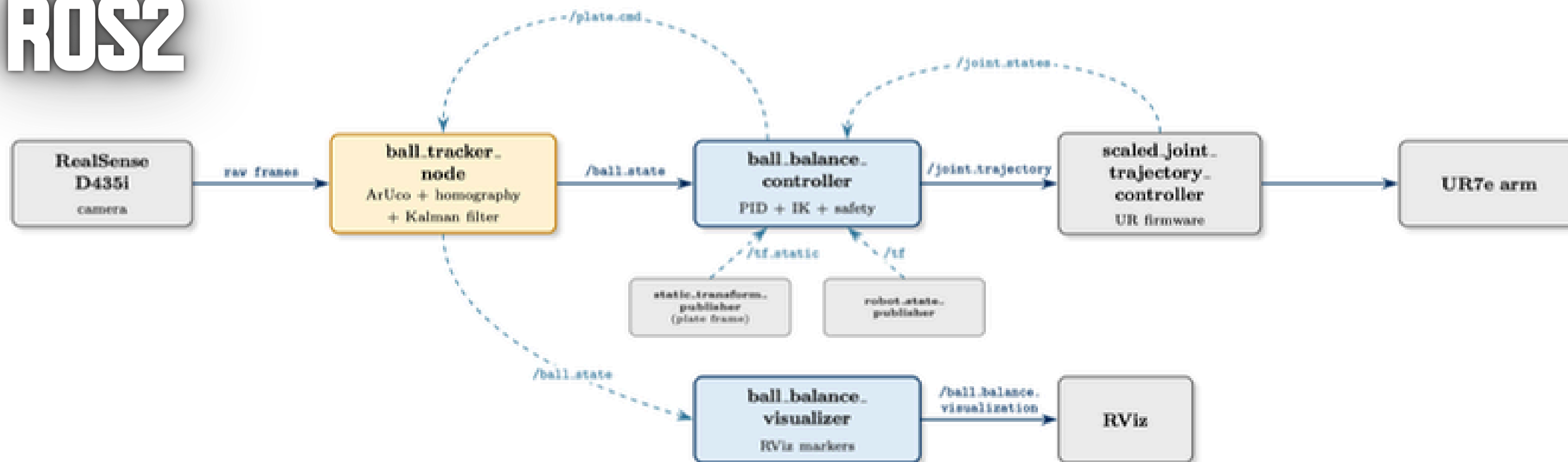
minimum-norm solution via Moore-Penrose pseudoinverse, computed by `numpy.linalg.lstsq`

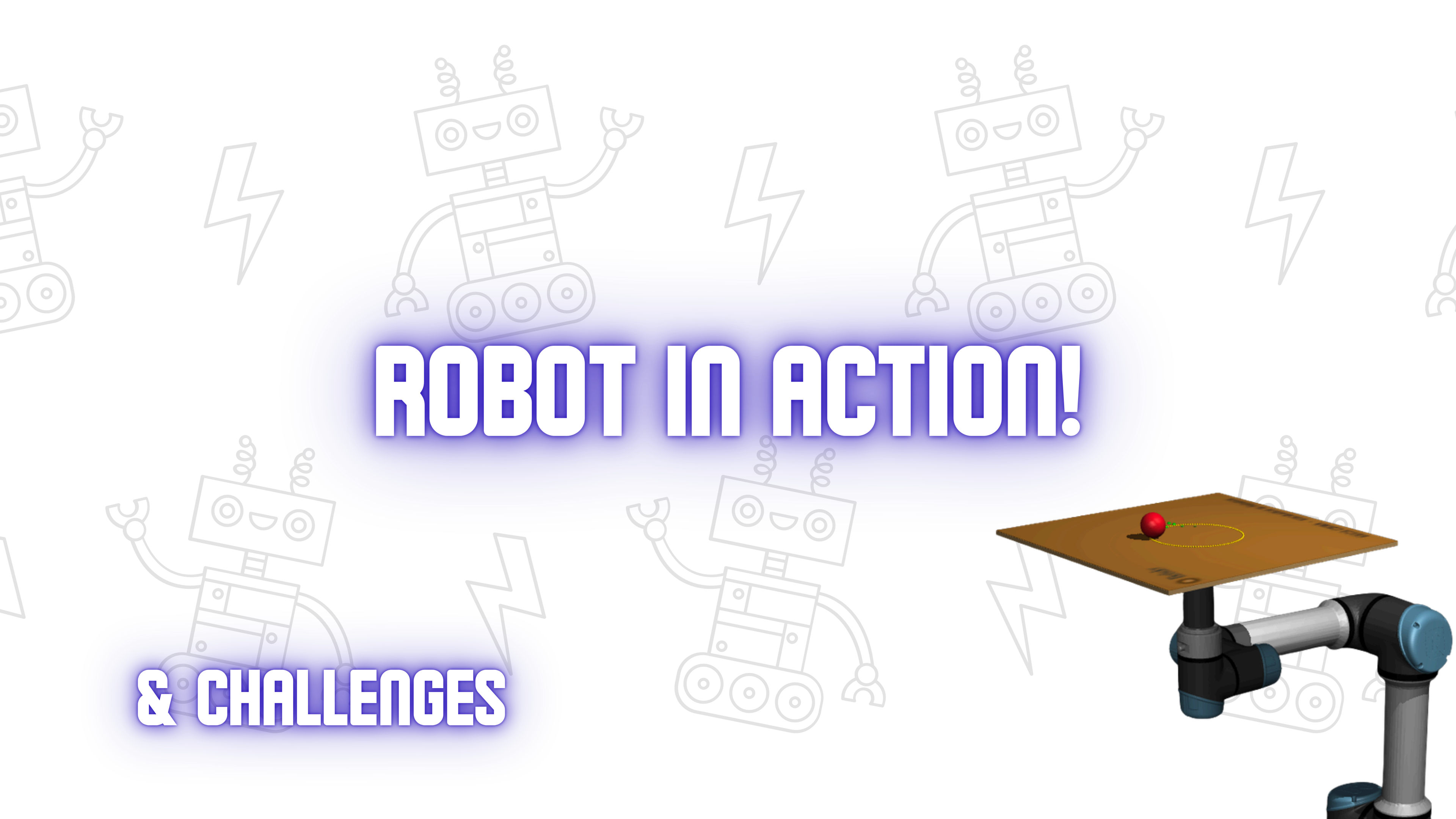


ACTUATION



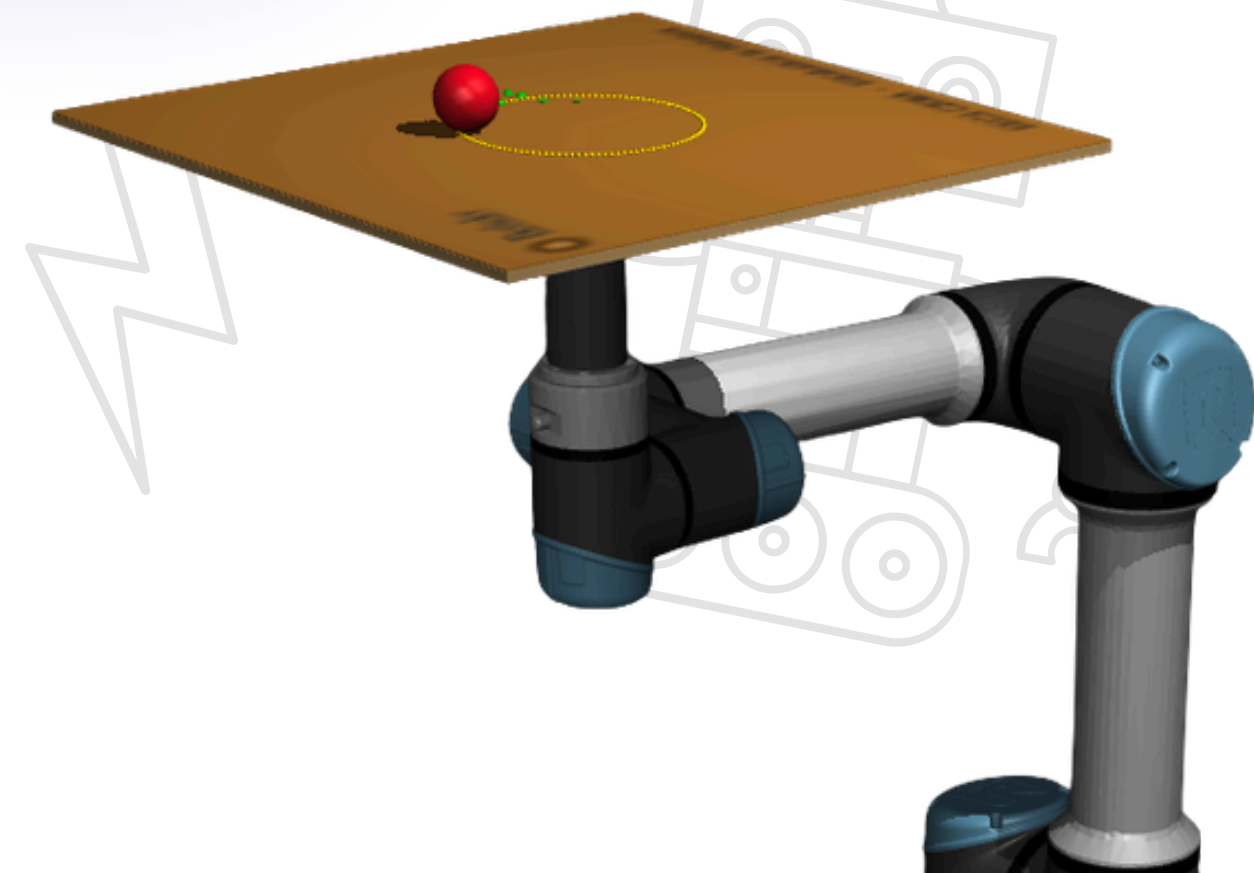
ROS2



The background features a repeating pattern of stylized, grey-outlined robots and lightning bolts. The robots have a boxy head with two circular eyes and a semi-circular mouth, a rectangular torso with a small square detail, and a base with three circular wheels. Their arms are raised in a 'V' shape. The lightning bolts are simple, jagged shapes. The entire pattern is set against a light blue gradient background.

ROBOT IN ACTION!

& CHALLENGES



CIRCLE TRAJECTORY

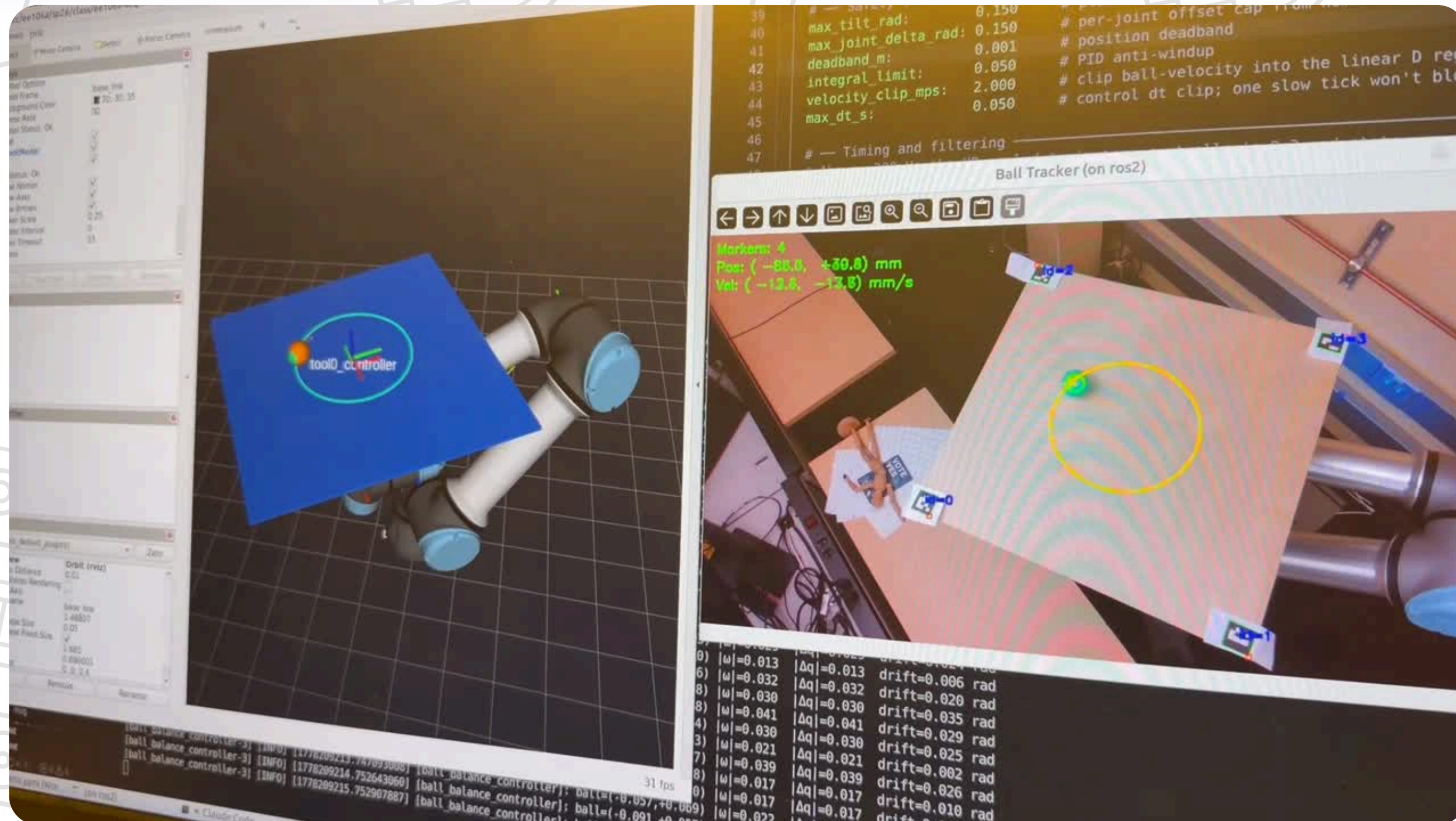
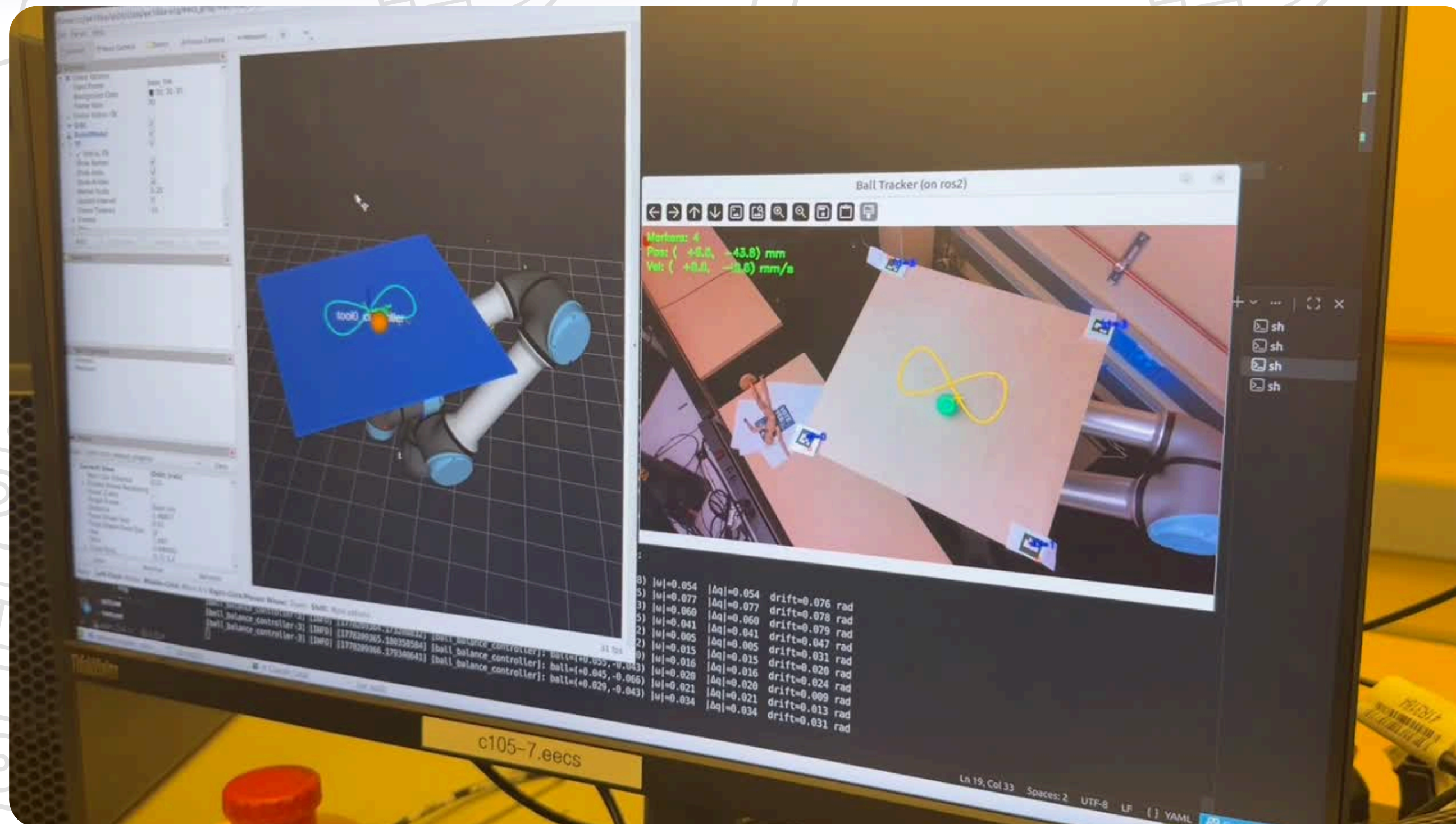
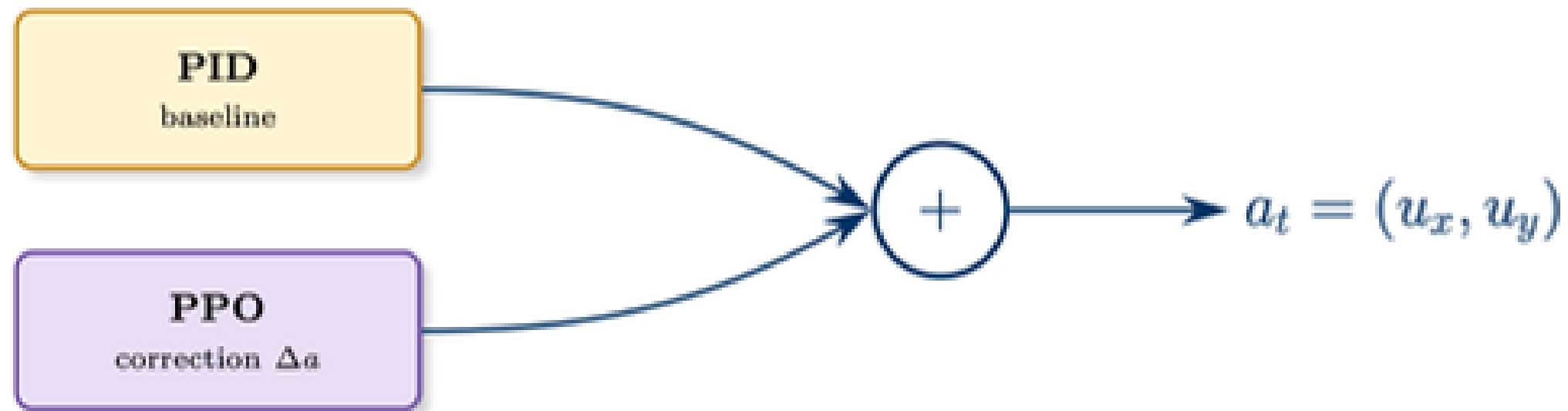


FIGURE-8 TRAJECTORY



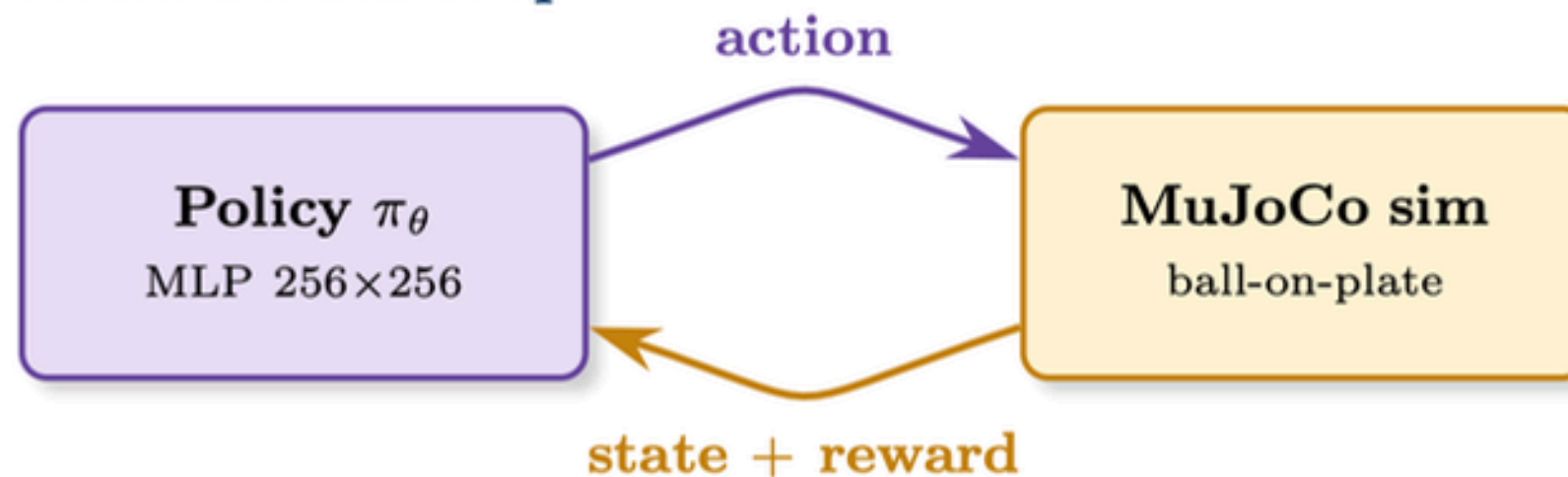
EXTENSION: RESIDUAL-PPO ON SIM

1. Residual control



The PID does most of the job; PPO learns small corrections that the PID alone cannot produce.

2. Standard RL loop



State (12-dim observation)

ball position	(x, y)
ball velocity	$(\dot{x}, \dot{y})$
target now	$(x_r, y_r, \dot{x}_r, \dot{y}_r)$
target lookahead	$(x_r^{+\Delta}, y_r^{+\Delta})$ at 0.2 s ahead
plate tilt	pitch, roll

Reward (per step)

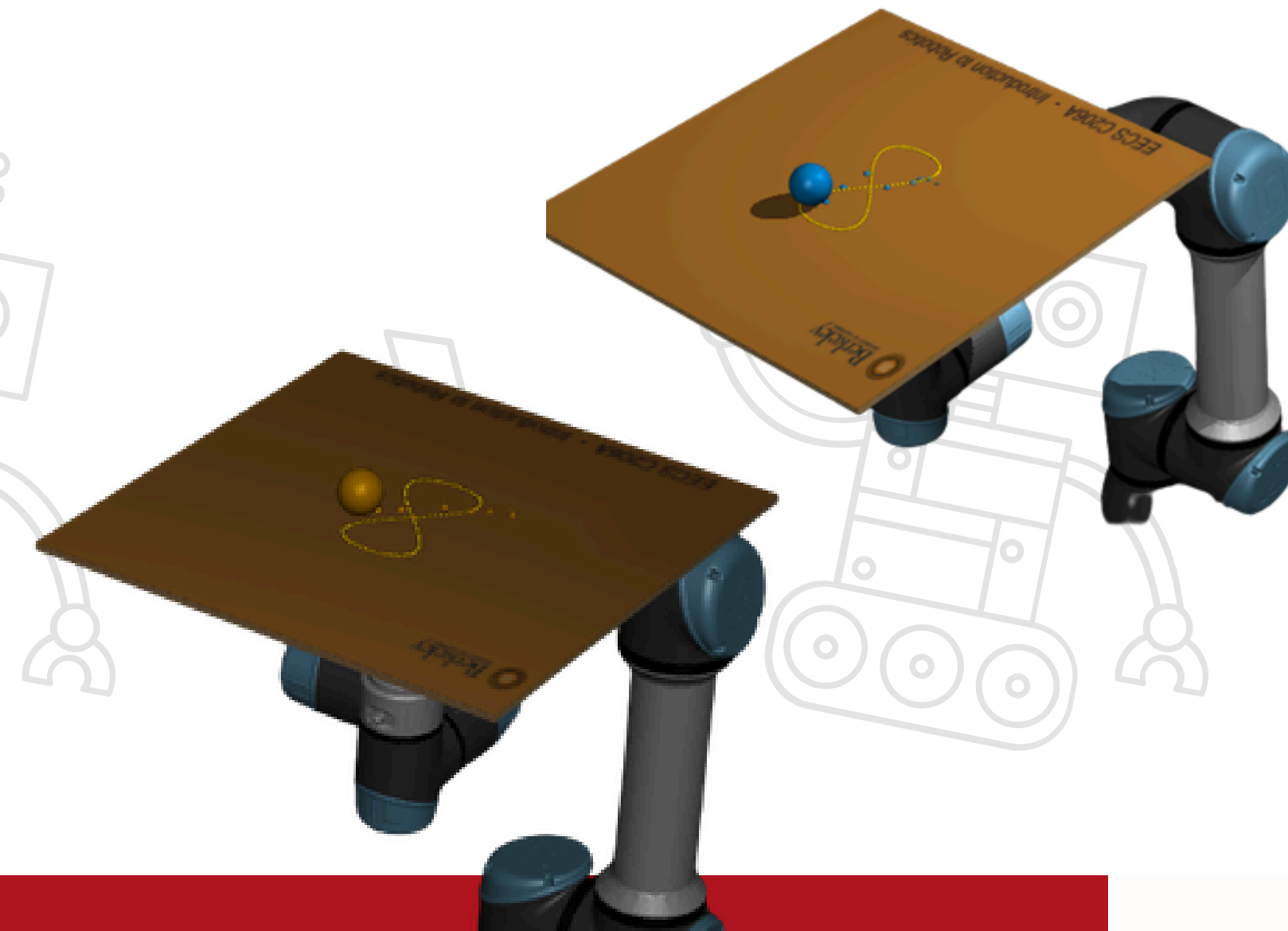
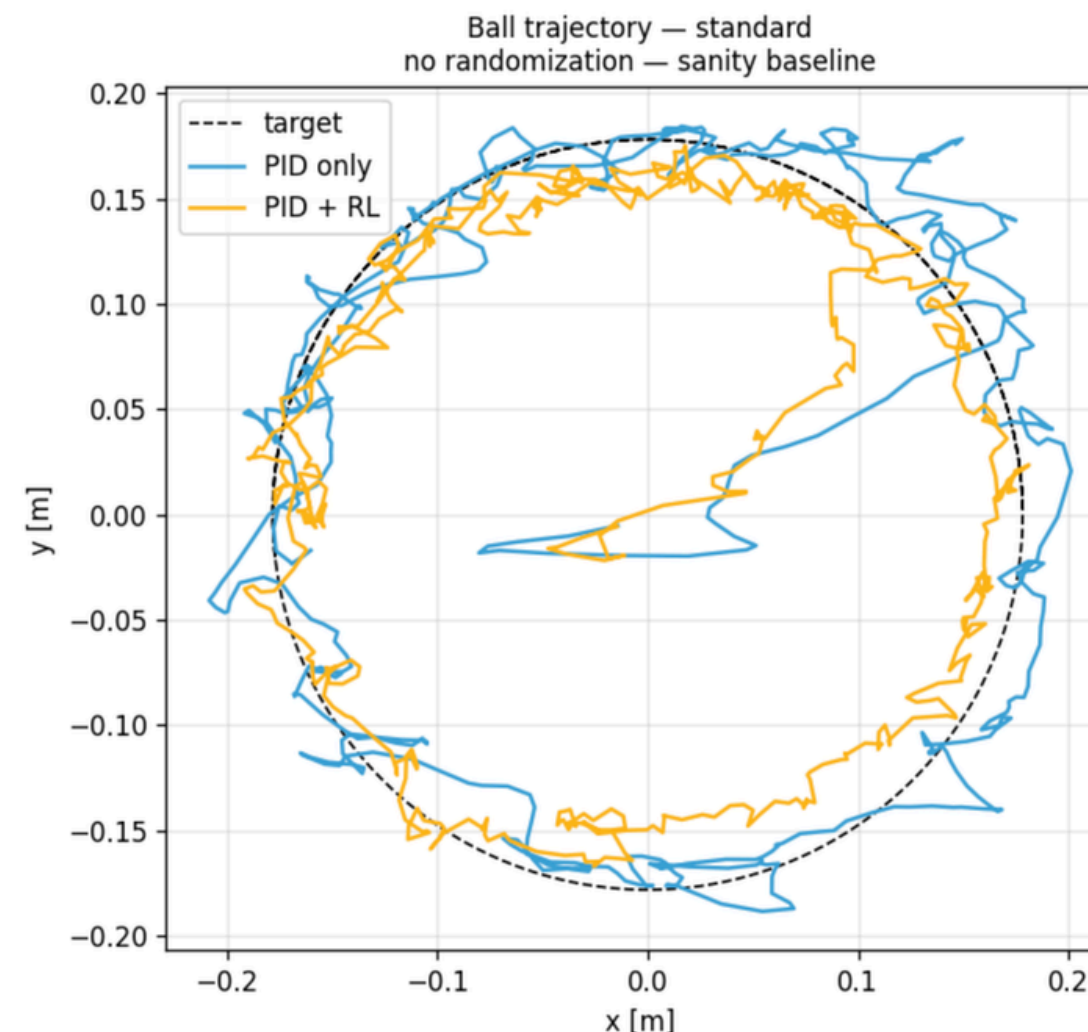
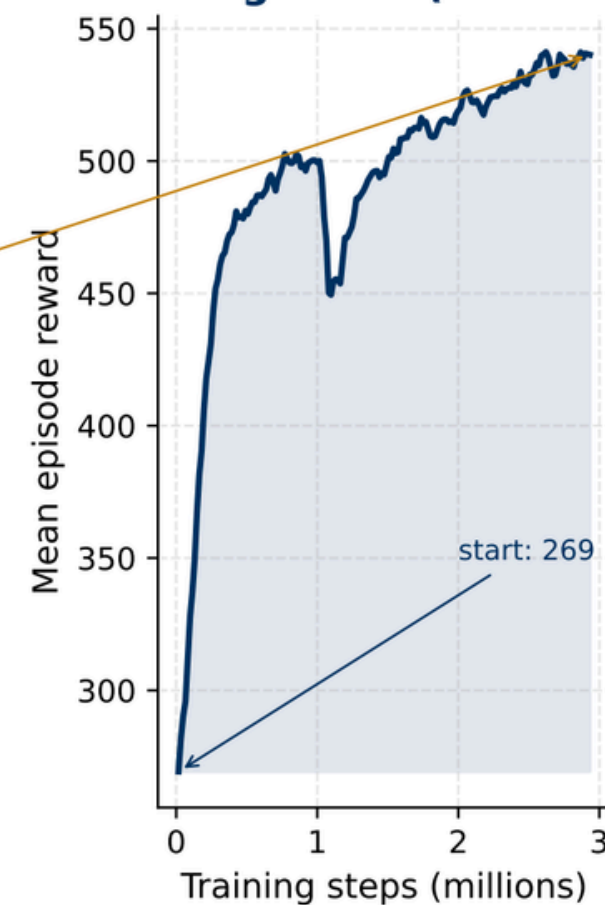
+	tracking	ball close to target
+	velocity match	$\dot{x} \approx \dot{x}_r$
+	progress	error decreasing
-	action magnitude	small $ a $ preferred
-	smoothness	avoid jitter
-	drop	big terminal penalty

RESIDUAL-PPO ON SIM

3. Domain randomization

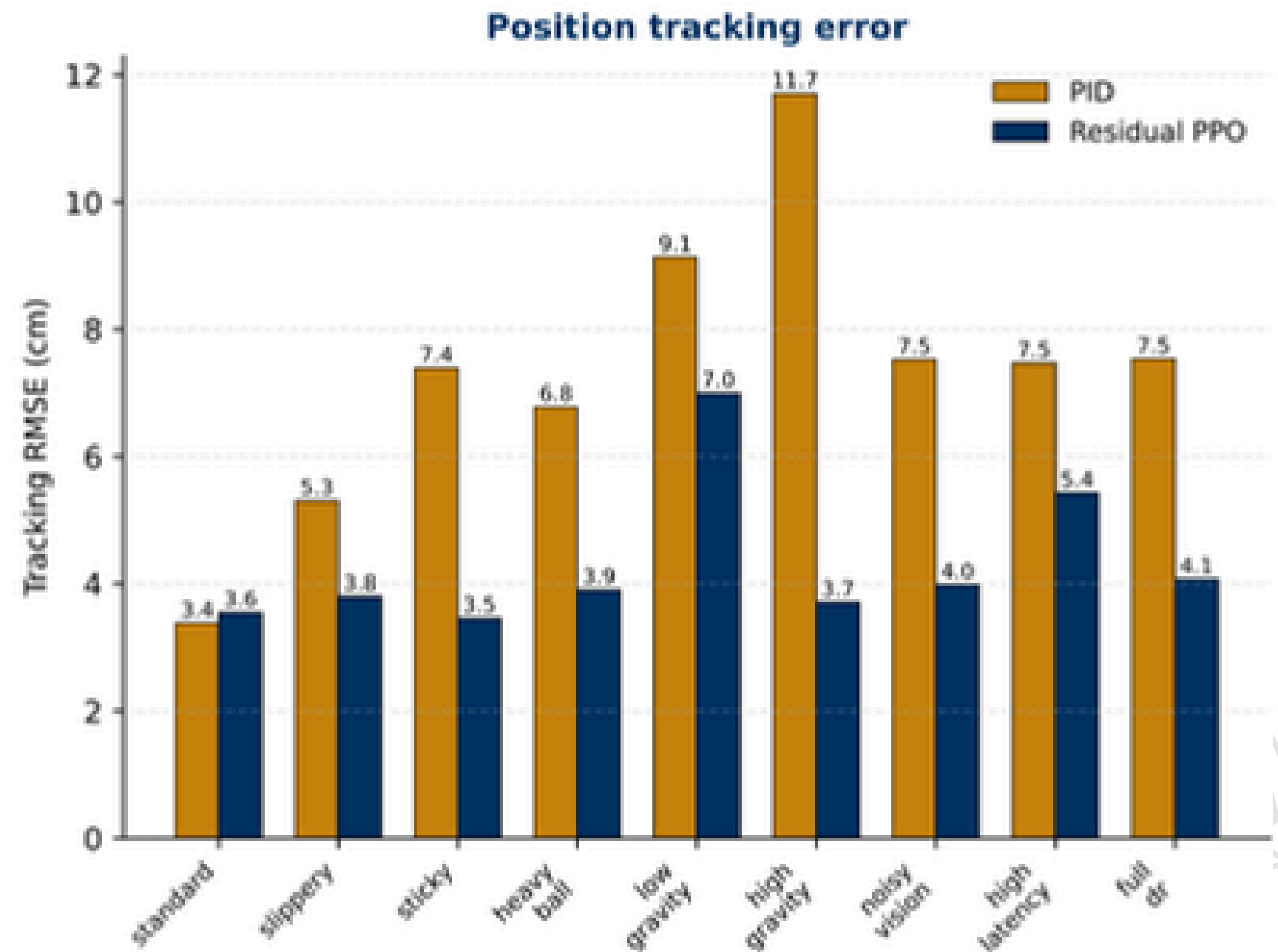
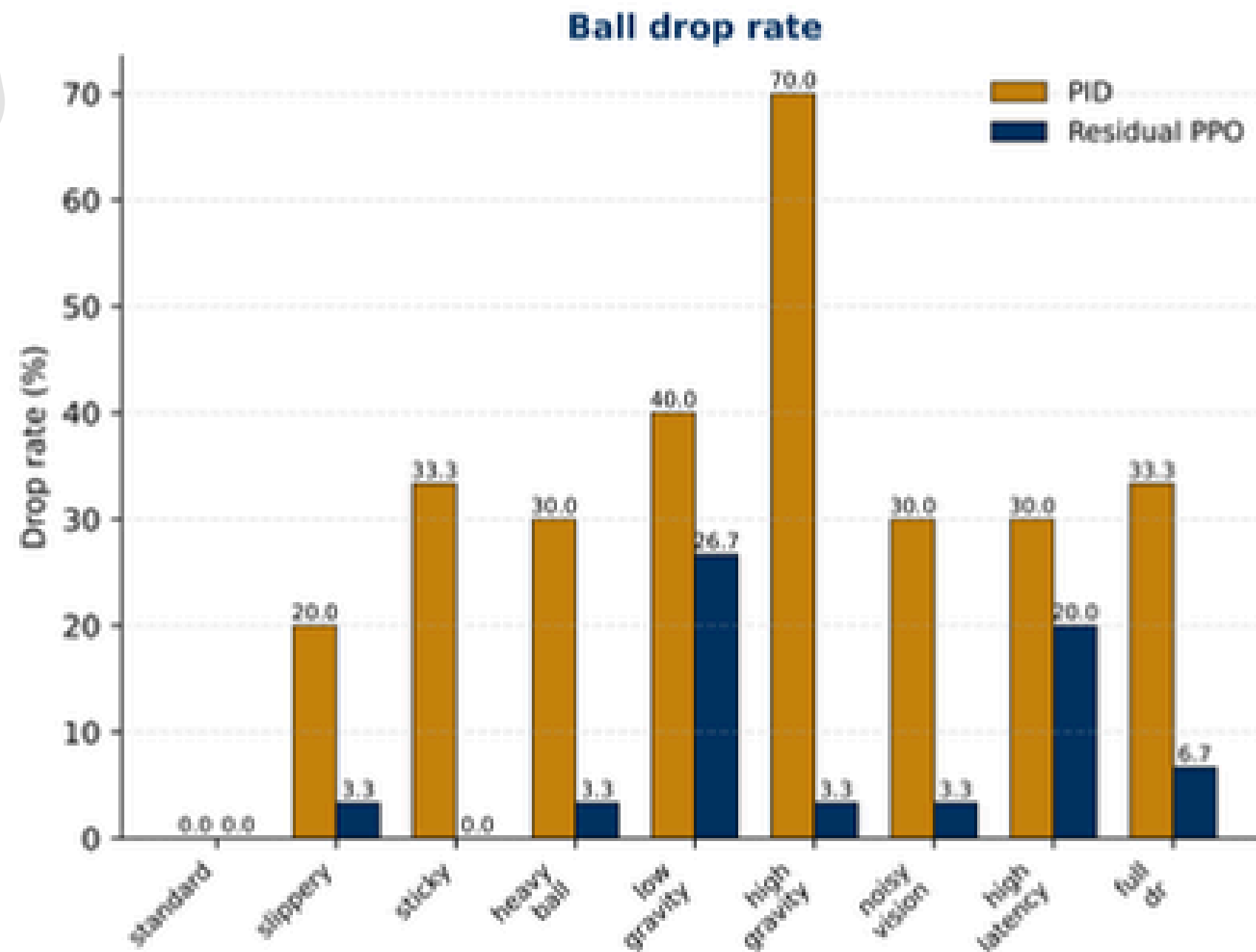
Each episode the simulator samples random values for the things we cannot measure exactly on the real robot. The policy must work across the whole range, so it stays robust when deployed.

- ball mass [2, 4] g
- initial ball offset [0, 2] cm
- ball friction [0.3, 0.7]
- observation noise 0–3 mm
- plate friction [0.3, 0.8]
- action delay [30, 150] ms
- gravity $\pm 2\%$



RESIDUAL-PPO ON SIM

PID vs Residual PPO — 10 domain conditions



THANK YOU

QUESTIONS?

